

MLOps Group Project

End-to-End MLOps Pipeline

IIT Jodhpur • PGD AI Program
Docker • GitHub Actions • Kaggle • W&B • Hugging Face

Task	Description	Marks
1	Set Up the GitHub Repository	10
2	Data Preparation & Normalisation	15
3	Select & Load a Model from Hugging Face	10
4	Train Multiple Versions on Kaggle & Track with W&B	25
5	Push Trained Model to Hugging Face Hub	5
6	Create a Dockerfile	10
7	Push to GitHub & Trigger GitHub Actions	15
8	Show All Experiments on W&B + Final Report	10
TOTAL		100

Note: The goal is not perfect accuracy — it is to demonstrate command of the full MLOps workflow. Marks come from how well you build, containerise, automate, and document the pipeline.

Task 1: GitHub Repository Setup

10 Marks

What to do

- Create a new public GitHub repository with README.md, .gitignore (Python), and MIT License.
- Create a develop branch. Protect main: require at least 1 PR review before merging.
- One team member is Admin (repo owner). All others added as Collaborators with Write access.
- Screenshot the Collaborators settings page for your report.

Step-by-step

1. Create the repo and folder structure

```
mkdir -p src .github/workflows data
touch src/train.py src/inference.py src/prepare_data.py
touch .github/workflows/ci.yml .github/workflows/inference.yml
touch requirements.txt
git add . && git commit -m "chore: init project structure"
git push origin main
```

2. Create develop branch & protect main

- On GitHub: Settings → Branches → Add branch protection rule for main
 - Check: Require a pull request before merging
 - Set Required approvals to 1
- Create develop branch: `git checkout -b develop && git push origin develop`

3. Add collaborators

- Settings → Collaborators → Add people → select Write role for each teammate
- One person keeps Admin (repo owner) — they do NOT need to be added separately

Checklist

- ☐ Public repo created with README, .gitignore, LICENSE
- ☐ develop branch created
- ☐ main branch protected (1 PR review required)
- ☐ All teammates added as Collaborators with Write access
- ☐ Screenshot of Collaborators page taken for report

Task 2: Data Preparation & Normalisation

15 Marks

Tip: Keep dataset under 50,000 samples for Kaggle free GPU. Recommended: IMDB Sentiment (25k, binary) or AG News (use 30k subset).

Step-by-step

1. Choose and load a dataset

```
from datasets import load_dataset

# Option A: IMDB Sentiment (binary: positive / negative)
dataset = load_dataset('imdb')

# Option B: AG News (4 classes: use subset)
dataset = load_dataset('ag_news')
dataset['train'] = dataset['train'].select(range(30000))
```

2. Inspect the raw data

```
import pandas as pd

train_df = pd.DataFrame(dataset['train'])
print(train_df.info())
print(train_df['label'].value_counts())
print("Nulls:", train_df.isnull().sum())
print("Duplicates:", train_df.duplicated().sum())
```

3. Clean the data (text)

```
import re

def clean_text(text):
    text = text.lower()
    text = re.sub(r'<.*?>', '', text) # remove HTML tags
    text = re.sub(r'[\w\s]', '', text) # remove punctuation
    text = text.strip()
    return text

train_df['text'] = train_df['text'].apply(clean_text)
train_df = train_df.drop_duplicates(subset='text')
train_df = train_df.dropna(subset=['text', 'label'])
```

4. Create id2label.json

```

import json

# For IMDb
id2label = {0: "negative", 1: "positive"}

# For AG News
# id2label = {0: "World", 1: "Sports", 2: "Business", 3: "Sci/Tech"}

label2id = {v: k for k, v in id2label.items()}

with open('data/id2label.json', 'w') as f:
    json.dump(id2label, f, indent=2)

print("Saved id2label.json")

```

5. Save cleaned data and update .gitignore

```

# Save locally (do NOT commit large files)
train_df.to_csv('data/train_clean.csv', index=False)

# Add to .gitignore:
# data/*.csv
# data/train_*.json
# !data/id2label.json  <-- this one IS committed

```

Checklist

- ☐ Dataset chosen (under 50k samples)
- ☐ Raw data inspected: size, class distribution, nulls, duplicates
- ☐ Text cleaned: lowercased, HTML removed, punctuation stripped, deduped
- ☐ id2label.json created and committed to repo
- ☐ Large data files added to .gitignore
- ☐ Data cleaning decisions documented in report (what + why)

Task 3: Select & Load a Model from Hugging Face

10 Marks

Tip: Use `distilbert-base-uncased` (67MB) for text classification. It retains 97% of BERT performance while being 40% smaller and 60% faster — ideal for free Kaggle GPU.

Step-by-step

1. Load the tokenizer

```
from transformers import AutoTokenizer

model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Test it
sample = tokenizer(
    "This movie was great!",
    truncation=True,
    padding="max_length",
    max_length=128
)
print(sample.keys())  # dict_keys(['input_ids', 'attention_mask'])
```

2. Load the model with your label mappings

```
from transformers import AutoModelForSequenceClassification
import json

with open("data/id2label.json") as f:
    id2label = json.load(f)
    id2label = {int(k): v for k, v in id2label.items()}

label2id = {v: k for k, v in id2label.items()}

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=len(id2label),
    id2label=id2label,
    label2id=label2id
)
print(f"Model loaded: {model.num_labels} labels")
```

3. Write the model rationale for your report (100-150 words)

Include these points in your report paragraph:

- Why this model (small size, strong baseline for text classification)
- Reference the HF model card: huggingface.co/distilbert-base-uncased

- Mention the size/speed tradeoff: 67MB, 40% smaller than BERT, 97% performance retained
- Suitability for the dataset/task chosen

Checklist

- ☐ Tokenizer loaded and tested
- ☐ Model loaded with correct num_labels from id2label
- ☐ 100-150 word model rationale written referencing the HF model card

Task 4: Train on Kaggle & Track with W&B

25 Marks

Warning: Training MUST happen on Kaggle Notebooks with GPU enabled — NOT inside GitHub Actions. Change only ONE hyperparameter between runs (e.g. `learning_rate`) for a meaningful W&B comparison.

Step-by-step

1. Create Kaggle Notebook & enable GPU

- Go to kaggle.com → New Notebook
- Settings → Accelerator → GPU T4 x2
- Set notebook to Public before submitting

2. Add secrets to Kaggle

- Kaggle → Account → Settings → Add Secrets
 - Add `WANDB_API_KEY` (from wandb.ai → Settings → API Keys)
 - Add `HF_TOKEN` (from huggingface.co → Settings → Access Tokens)
- Never paste raw tokens directly in notebook cells

3. Load secrets at the top of the notebook

```
from kaggle_secrets import UserSecretsClient
import os, wandb
from huggingface_hub import login

secrets = UserSecretsClient()
os.environ['WANDB_API_KEY'] = secrets.get_secret('WANDB_API_KEY')
login(token=secrets.get_secret('HF_TOKEN'))
wandb.login()
```

4. Define compute_metrics

```
from sklearn.metrics import accuracy_score, f1_score

def compute_metrics(pred):
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1)
    return {
        'accuracy': accuracy_score(labels, preds),
        'f1': f1_score(labels, preds, average='weighted'),
    }
```

5. Version 1 — Baseline (lr=3e-5, batch=16, epochs=3)

```

import wandb
from transformers import TrainingArguments, Trainer

wandb.init(
    project='mlops-assignment3',
    name='run-v1',
    config={
        'model': model_name,
        'epochs': 3, 'batch_size': 16,
        'learning_rate': 3e-5, 'version': 'v1',
        'platform': 'Kaggle'
    }
)

training_args = TrainingArguments(
    output_dir='./results-v1',
    num_train_epochs=3,
    per_device_train_batch_size=16,
    eval_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
    report_to='wandb',
    run_name='run-v1',
    learning_rate=3e-5
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics,
)
trainer.train()
wandb.finish()

```

6. Version 2 — Changed hyperparameter (lr=5e-5)

Fork the notebook, change only learning_rate, update run name to run-v2:

```

# Change ONLY learning_rate (or batch_size or epochs)
wandb.init(
    project='mlops-assignment3',
    name='run-v2',
    config={
        'model': model_name,
        'epochs': 3, 'batch_size': 16,
        'learning_rate': 5e-5, 'version': 'v2', # <-- CHANGED
        'platform': 'Kaggle'
    }
)

training_args = TrainingArguments(
    ...
    learning_rate=5e-5, # <-- CHANGED
    run_name='run-v2',
)
# Rest of training code is the same

```


7. Set W&B project to Public

- wandb.ai → your project → Settings → Privacy → Public
- Copy the dashboard URL for your report
- Screenshot the Runs Comparison table showing both runs

Checklist

- ☐ Kaggle notebook created, GPU T4 enabled
- ☐ WANDB_API_KEY and HF_TOKEN stored as Kaggle Secrets
- ☐ Version 1 trained and logged to W&B (accuracy, F1, loss, hyperparams)
- ☐ Version 2 trained with one changed hyperparameter
- ☐ Both runs visible in W&B dashboard
- ☐ W&B project visibility set to Public
- ☐ W&B dashboard screenshot taken for report
- ☐ Both Kaggle notebooks set to Public

Task 5: Push Model to Hugging Face Hub

5 Marks

Step-by-step

1. Push model and tokenizer from Kaggle

```
from huggingface_hub import login

login(token=secrets.get_secret('HF_TOKEN'))

# Replace with your HF username and repo name
MODEL_REPO = "your-username/mlops-imdb-distilbert"

model.push_to_hub(MODEL_REPO)
tokenizer.push_to_hub(MODEL_REPO)
print(f"Model pushed to: https://huggingface.co/{MODEL_REPO}")
```

2. Log the HF URL to your W&B run summary

```
wandb.run.summary['huggingface_model'] = \
    f"https://huggingface.co/{MODEL_REPO}"
wandb.finish()
```

3. Verify and set to Public

- Go to huggingface.co → your model repo → Settings → set to Public
- Copy the URL for your report

Checklist

- ☐ Model and tokenizer pushed to Hugging Face Hub
- ☐ HF model URL logged to W&B run summary
- ☐ HF repo set to Public
- ☐ HF model URL noted for report

Task 6: Create a Dockerfile

10 Marks

Step-by-step

1. Write the Dockerfile (in repo root)

```
FROM python:3.11-slim

WORKDIR /app

# Accept HF model name as build argument with a default
ARG HF_MODEL_NAME=your-username/mlops-imdb-distilbert
ENV HF_MODEL_NAME=$HF_MODEL_NAME

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/inference.py .

CMD ["python", "inference.py"]
```

2. Write src/inference.py

```
import os
from transformers import pipeline

hf_model = os.environ.get("HF_MODEL_NAME")
hf_token = os.environ.get("HF_TOKEN")
input_text = os.environ.get("INPUT_TEXT", "This movie was great!")

classifier = pipeline(
    "text-classification",
    model=hf_model,
    token=hf_token
)

result = classifier(input_text)
print(f"Input: {input_text}")
print(f"Result: {result}")
```

3. Write requirements.txt

```
transformers>=4.40.0
torch>=2.0.0
datasets>=2.18.0
scikit-learn>=1.4.0
wandb>=0.16.0
huggingface_hub>=0.22.0
```

4. Build and test locally

```
# Build
docker build \
  --build-arg HF_MODEL_NAME=your-username/mlops-imdb-distilbert \
  -t mlops-a3-inference:latest .

# Test
docker run --rm \
  -e HF_TOKEN=<your_hf_token> \
  -e INPUT_TEXT="This movie was fantastic!" \
  mlops-a3-inference:latest
```

5. Push to Docker Hub

```
docker login

docker tag mlops-a3-inference:latest \
  your-dockerhub-username/mlops-a3-inference:latest

docker push your-dockerhub-username/mlops-a3-inference:latest
```

Tip: Create a free Docker Hub account at hub.docker.com. Create a public repository first, then push. The image URL will be: `docker.io/your-username/mlops-a3-inference:latest`

Checklist

- ☐ Dockerfile written with `python:3.11-slim` base image
- ☐ `HF_MODEL_NAME` accepted as ARG with a default value
- ☐ `inference.py` reads model, token, and input from environment variables
- ☐ `requirements.txt` contains only inference dependencies
- ☐ Image built and tested locally — inference runs end-to-end
- ☐ Image pushed to Docker Hub as a public repo
- ☐ Docker image URL noted for report

Task 7: GitHub Actions Workflows

15 Marks

Warning: GitHub Actions is for CI (linting) and inference ONLY. Training must happen on Kaggle. Do NOT run model training inside GitHub Actions workflows.

7.1 Add GitHub Secrets

- Settings → Secrets and Variables → Actions → New repository secret
- Add HF_TOKEN (from Hugging Face)
- Add WANDB_API_KEY (from W&B)
- Never commit these values to any file in the repo

7.2 CI Workflow — triggers on push to develop

Create .github/workflows/ci.yml:

```
name: CI
on:
  push:
    branches: [develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt flake8
      - run: flake8 src/ --max-line-length=120
```

7.3 Inference Workflow — triggers manually

Create .github/workflows/inference.yml:

```
name: Inference
on:
  workflow_dispatch:
    inputs:
      input_text:
        description: 'Text to classify'
        required: true

jobs:
  infer:
    runs-on: ubuntu-latest
```

```
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with:
      python-version: '3.11'
  - run: pip install -r requirements.txt
  - env:
      HF_TOKEN: ${ secrets.HF_TOKEN }
      INPUT_TEXT: ${ github.event.inputs.input_text }
    run: python src/inference.py
```

7.4 Trigger and verify

- Push to develop → go to Actions tab → CI → verify green checkmark
- Actions tab → Inference → Run workflow → enter test text → verify green run
- Screenshot the successful inference Actions log for your report

Checklist

- ☐ HF_TOKEN and WANDB_API_KEY added to GitHub repository Secrets
- ☐ ci.yml created — triggers on push to develop / PR to main
- ☐ inference.yml created — triggers via workflow_dispatch
- ☐ CI run passes (green) after pushing to develop
- ☐ Inference workflow triggered manually and produces correct output
- ☐ Screenshot of successful Actions run taken for report

Task 8: W&B Dashboard & Final Report

10 Marks

W&B Dashboard

- Verify both run-v1 and run-v2 appear in wandb.ai with all metrics logged
- Use Runs Comparison table: select both runs → compare accuracy, F1, loss side by side
- Project Settings → Privacy → Public — copy the URL

Report Requirements (4-5 pages, PDF)

Must include:

- Student names and individual contributions
- All live public links: GitHub, Kaggle (one per experiment), HF Model, Docker image, W&B
- Screenshot: GitHub Collaborators settings page
- Data cleaning decisions — what you did and why (missing values, normalisation, class distribution)
- Model selection rationale (100-150 words referencing HF model card)
- Experiment comparison table: Version 1 vs Version 2 metrics, which performed better and why
- Screenshot or badge showing successful GitHub Actions inference run
- Docker design: describe Dockerfile choices, paste Actions log from successful inference run
- Challenges & learnings: what was hard, what you would do differently

Experiment comparison table template:

Metric	Version 1 (lr=3e-5)	Version 2 (lr=5e-5)	Winner
Accuracy	fill in	fill in	—
F1 Score	fill in	fill in	—
Val Loss	fill in	fill in	—
Train Loss	fill in	fill in	—

Critical: All links in your report must be live and publicly accessible at submission time. Test every link in a private/incognito browser before submitting.

Final Checklist

- ☐ Both W&B runs visible with accuracy, F1, and loss metrics
- ☐ Runs Comparison table screenshot taken
- ☐ W&B project set to Public, URL copied

- ❑ All 5 public links verified in incognito browser
- ❑ 4-5 page PDF report written with all required sections
- ❑ PDF submitted via course portal

Good luck! — Focus on the pipeline, not the accuracy.